A reflexive-base introduction rule for iPRA

A six-rule presentation with ordinary closure semantics

Research report prepared for Vladimir Reshetnikov

11 November 2026

Abstract

Replace the current introduction rule's test on the syntactic form of its existential witness by a test on its zero instance. Use equation progress exactly when that instance is a syntactically reflexive equation; use numerical cases otherwise. On the existing zero-normal term syntax this yields six primitive rules and exactly the sequents of the repository's zero-normal kernel. Missing case steps are recovered by cutting out a proved reflexive base, and arbitrary equation induction is recovered through a selector gate. The guard is preserved, but need not be reflected, by substitution; the latter issue is handled explicitly. A raw-term version works without normalization and has seven rules. Both versions inherit the complete bi-interpretation with intuitionistic primitive recursive arithmetic, with whole numerical domains, actual equality, and identity numerical composites. The matched strict checker decreases from 3,095 to 3,084 normalized Python bytes and from 1,075 to 1,073 AST nodes; its 1,013-token count is unchanged. This is a small, concrete simplification, not a global minimality theorem or a large new checker-size record.

1 Result and comparison boundary

The comparison is pinned to ProveIt commit `d9383325b18d349b8bfdc54ef564c5784cc029ec`, under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The inspected README lists forty-nine research packages and the associated Lean formalization [1]. At that snapshot the ordinary kernel `NPrf` has eight rules, the witness-selected form has seven, and the formally proved zero-normal kernel `NZPrf` has seven. The written `zero_normal` package already combines zero normalization with the witness-selected introduction to obtain six rules [2].

The new proposal keeps that six-rule count. Its change is the introduction criterion

$$B(t, 0) \text{ is syntactically reflexive,}$$

in place of the joint requirement that the matrix is an equation and the witness is the literal zero. The term language, the meanings of the logical binders, and the numerical domain stay the same. This criterion directly identifies bases that can be proved before using any induction.

Main theorem. For every normal context and formula,

$$\Gamma \vdash_{\mathcal{R}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{Z}} A.$$

The proposed calculus is equiconsistent and bi-interpretable with full first-order iPRA. The arithmetic interpretations have the whole domain as their domain formula, actual equality, and numerical composite graph $x = y$.

There are two separate simplicity claims. Mathematically, the rule no longer requires a distinguished syntactic representative of the witness or the construction of an alternative term denoting zero to recover a missing case. Operationally, the base test reuses a formula the checker already needs, and a validated-arity test gives a small matched source reduction. The source reduction is only eleven bytes; calling it a major breakthrough would overstate the result.

The construction does not beat the repository's separate 516-byte compact guarded codec. That codec has a different certificate representation and public syntax contract. The report does not conflate a fully validated tagged checker with that smaller profile.

1.1 What is new and what is inherited

The zero-normal syntax, crossed one-term transport, numerical and logical bridge to iPRA, and the general guarded-introduction strategy are inherited. The specific reflexive-base test, its proof allowing a false guard to become true under substitution, and its raw-term asymmetric selector gate are the contribution of this report.

The cumulative rule retaining both the zero and current induction assumptions is already present as *Old DI* in `inherited_choice_basis.tex`. It is not claimed as new here. Neither deletion of the zero computation rule nor the six-rule count is claimed as new. The comparison is against the pinned source, not an unverified reconstruction of an earlier draft.

1.2 The target theory

Here iPRA means full intuitionistic first-order logic with equality, in a language containing a symbol for each finite primitive-recursive description: zero, successor, projections, composition, and primitive recursion. Its nonlogical axioms are the corresponding defining equations, successor separation and injectivity, and induction for quantifier-free formulas, with parameters and retained assumptions. This is the target defined in the repository's `PR.lean`; it is not merely the equational fragment or a statement about standard natural-number evaluations.

Contents

1 Result and comparison boundary **1**

1.1 What is new and what is inherited 2

1.2 The target theory 2

2 Introduction selected by a reflexive base **4**

2.1 Normal syntax and the complete rule inventory 4

2.2 The noncircular structural bootstrap 5

2.3 Recovering progress by equational decoders 6

2.4 Exact equivalence and bi-interpretation 8

2.5 The same guard on raw syntax 8

2.6 Further variants and the implementation tradeoff 9

3 The interpretations and both composites **10**

3.1 The established scalar bridge 10

3.2 Explicit composition for the new theory 10

4 Complete checker and input contract **11**

4.1 Syntax and variables 11

4.2 The six instructions 12

4.3 Why the short base test is exact 12

4.4 Soundness and completeness of the checker interface 12

5 Measurements and verification **13**

6 Further research questions **13**

7 Reproduction and deliverables **14**

A The entire selected checker **14**

2 Introduction selected by a reflexive base

The native zero-normal calculus admits a single introduction whose selection test uses the formula it would otherwise retain as the base. It requires no distinguished witness and no syntactically disguised zero. The construction is an instance of the guarded-introduction method, with a useful difference: its guard need only be preserved by substitution; it need not be reflected. A newly reflexive base permits an elementary repair of the substituted inference.

2.1 Normal syntax and the complete rule inventory

Let nf be the existing structural zero normalization: it recursively normalizes the children of a term, then replaces $I(0, a; u.h)$ by a. It performs no successor computation. Terms and formulas in this section are zero-normal. Every displayed substitution means capture-free substitution followed by nf on all numerical terms. Write $B(t, r)$ for this normal instance of a matrix whose distinguished coordinates are x, y. The formula constructors are still equality, Q, and D; their ordinary readings are unchanged:

$$Q_y B = B(0) \to \forall y\, B(y), \qquad D_{x,y} B = \exists x (B(x, 0) \to \forall y\, B(x, y)).$$

The numerical syntax is $x, 0, S(t), I(n, a; u.h)$, with the literal-zero counter excluded from a normal iterator node. The equation $0 = S0$ is the designated contradiction.

Define the decidable metasyntactic predicate

$$\mathsf{R}(B, t) \quad \Longleftrightarrow \quad B(t, 0) \text{ is literally } r = r \text{ for some normal term } r.$$

This is a comparison of two finite syntax trees, not a call to arithmetic equality or a theorem prover. Since substitution preserves the formula root, $\mathsf{R}(B, t)$ entails that B is a literal equation. Set

$$\eta_{B,t}(z) = \begin{cases} z, & \mathsf{R}(B, t), \\ 0, & \text{otherwise.} \end{cases}$$

The calculus $\mathcal{R}$ has exactly these six primitive rules:

$$\frac{A \in \Gamma}{\Gamma \vdash A} \qquad \text{Hyp,}$$

$$\Gamma \vdash I(Sn, a; u.h) = h\big(I(n, a; u.h)\big) \qquad \text{IS,}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)} \qquad \text{Tr,}$$

$$\frac{\Gamma \vdash Q_y B \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad \text{QE,}$$

$$\frac{\Gamma \vdash D_{x,y} B \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A} \qquad \text{DE,}$$

$$\frac{\Gamma, B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B} \qquad \text{DI}_{\mathsf{R}}.$$

All terms in IS and Tr are understood in the normal syntax, including the possible contraction of $I(n, a; u.h)$ when n is literally zero. In DE, b is fresh for the retained context, conclusion, and fixed parameters. In DI, z is fresh for the retained context, witness, and fixed parameters. No other equality, logical, structural, induction, or computation rule is primitive.

For comparison let $\mathcal{Z}$ be the repository's seven-rule zero-normal calculus: it has the first five rules above and, instead of DI_{R}, has unrestricted D introduction by cases and Q introduction by equation progress:

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B}, \qquad \frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)}.$$

In the second rule E is a literal equation. The repository proves that $\mathcal{Z}$ derives the normal form of exactly the native sequents.

2.2 The noncircular structural bootstrap

Normalization facts used. The existing normalization layer proves idempotence and compatibility with substitution. In particular the normal form of a substitution instance depends only on the normal forms of the term and the substitution's values. Thus normal substitution is associative with the usual lifted substitutions under binders. These facts are syntactic theorems; they do not use any rule of $\mathcal{R}$.

Weakening, renaming, and cut come first. Weakening, exchange, contraction, injective renaming of free variables, and simultaneous ordinary-assumption cut are admissible by induction on derivations. These operations leave the matrix and witness unchanged, or apply an injective variable renaming. A reflexive equation stays reflexive and a nonreflexive equation cannot become reflexive under such a renaming. Hence no DI mode changes in these inductions. At rules with eigenvariables first choose fresh names away from the inserted derivations. This argument does not yet invoke arbitrary numerical substitution.

Equality comes before missing cases. For any normal term r, let

$$J_r := I(S0, r; u.u).$$

IS gives $J_r = r$: its right-hand side is the zero-normal form of $I(0, r; u.u)$, namely r. Apply Tr with identity template and with this same equation as both premises. The conclusion is $r = r$. From a proof of $s = t$, Tr with identity template, major premise $s = s$, and minor premise $s = t$ gives $t = s$. Tr then gives transitivity and term congruence. These constructions use no introduction rule and no induction. In particular, every base singled out by R is already provable in the retained context.

Lemma 2.1 (Every case introduction is admissible)**.** *For every normal native matrix B and witness t,*

$$\Gamma, B(t,0) \vdash_{\mathcal{R}} B(t,Sz) \quad \Longrightarrow \quad \Gamma \vdash_{\mathcal{R}} D_{x,y}B.$$

Proof. When $\mathsf{R}(B,t)$ is false, this is primitive DI. When it is true, the discharged base is literally $r = r$. Use the equality proof above and ordinary-assumption cut to remove this base from the supplied derivation. Weaken the resulting derivation of $B(t,Sz)$ by $B(t,z)$, then apply the progressive mode of primitive DI. No substitution or general progress rule is used. □

Lemma 2.2 (One-way preservation is sufficient)**.** *Arbitrary capture-free normal numerical substitution is admissible in $\mathcal{R}$.*

Proof. If $B(t,0)$ is literally $r = r$, applying any normal substitution yields $\mathrm{nf}(r\sigma) = \mathrm{nf}(r\sigma)$. The compatibility laws identify this with the zero instance of the substituted matrix and witness. Thus a true guard remains true.

Now induct on a derivation. IS, Tr, QE and DE commute with normal substitution by the normalization facts and the standard fresh-name convention. In a progressive DI step the guard was true and remains true, so the induction hypothesis is the exact premise of progressive DI at the substituted matrix and witness. In a case DI step the induction hypothesis is the substituted case premise. Apply Lemma 2.1, whether or not its guard has newly become true. This completes the induction. □

The reflection direction genuinely fails and is unnecessary. For example, an equation with zero instance $p = q$ is initially nonreflexive when p, q are distinct parameters, but becomes reflexive after $p := 0, q := 0$. The substitution proof repairs precisely that change; it does not falsely claim that every rule label is preserved.

The dependency order so far is normalization laws; weakening and cut; elementary equality; unrestricted case introduction; arbitrary substitution. None of these steps presupposes the equivalence theorem.

2.3 Recovering progress by equational decoders

The general equation-progress rule can be recovered without constructing an implication macro and without substituting numerical terms into the supplied step derivation. This shorter proof uses only equality, case-derived closure, weakening, renaming, and ordinary-assumption cut. All formulas below are zero-normal, and substitutions into formulas mean the existing normal substitutions.

The needed case-derived closure rule is

$$\frac{\Lambda, F(0) \vdash F(Sz)}{\Lambda \vdash Q_y F(y)}, \tag{1}$$

where z is fresh for Λ and the fixed parameters. It is already available from unrestricted case DI, followed by DE with an unused witness. Unrestricted case DI itself follows from the reflexive-base introduction: if its guard is true, prove the reflexive base, cut it out, and weaken by the progressive hypothesis. Neither construction needs general equation progress or arbitrary numerical substitution on proofs.

Let

$$C(v, a, b) := I(v, a; u.b), \qquad u \notin \mathrm{FV}(b).$$

Its zero equation is definitional and its successor equation is a direct instance of the remaining computation rule.

Lemma 2.3 (Case decoder)**.** *Let $r(y)$ be a normal term template and let a be a term independent of y. Define*

$$D_{r,a}(y) := \big(C(y, a, C(y, 0, r(y))) = r(y)\big).$$

Then $Q_y D_{r,a}(y)$ is derivable without assumptions. Consequently

$$\Lambda \vdash a = r(0) \quad \Longrightarrow \quad \Lambda \vdash D_{r,a}(v)$$

for every term v.

Proof. The zero instance of $D_{r,a}$ is literally $a = r(0)$ in zero-normal syntax. At $y = Sz$, two successor computations and transitivity prove

$$C(Sz, a, C(Sz, 0, r(Sz))) = C(Sz, 0, r(Sz)) = r(Sz).$$

This proof ignores the temporary base hypothesis. Rule (1) therefore gives $Q_y D_{r,a}(y)$ in the empty context. Weaken this closure into Λ and apply QE with the given base equation and target v. □

This is a valid fixed-context numerical case argument. The context Λ is held fixed and has no occurrence of the fresh generalized variable z. The conclusion may later be instantiated at a variable that appears in other hypotheses because QE, unlike the introduction rule, imposes no freshness condition on its numerical target.

Theorem 2.4 (Unrestricted equation progress)**.** *Suppose the supplied derivation is*

$$\pi : \quad \Gamma, E(z) \vdash E(Sz), \qquad E(y) \equiv s(y) = t(y),$$

with z fresh for Γ and the fixed parameters. Then the reflexive-base calculus derives $\Gamma \vdash Q_y E(y)$.

Proof. Write $s_0 = s(0)$, $t_0 = t(0)$, and temporarily retain the original base:

$$\Delta := \Gamma, E(0) = \Gamma, (s_0 = t_0).$$

The progressive lift is

$$H(y) := \big(C(y, 0, s(y)) = C(y, 0, t(y))\big).$$

Its zero-normal base is literally $0 = 0$, so it qualifies for primitive progressive DI. Define the two decoder equations

$$\begin{aligned} D_s(y) : &\quad C(y, s_0, C(y, 0, s(y))) = s(y), \\ D_t(y) : &\quad C(y, s_0, C(y, 0, t(y))) = t(y). \end{aligned}$$

Lemma 2.3 gives their closures without assumptions. For D_s the zero base is $s_0 = s_0$, supplied by reflexivity; for D_t it is $s_0 = t_0$, supplied by the retained hypothesis $E(0)$. Hence, in the fixed context Δ, QE gives both $D_s(z)$ and $D_t(z)$ at the fresh variable z.

Now add $H(z)$. Congruence in the one-term context

$$w \longmapsto C(z, s_0, w)$$

converts it to

$$C(z, s_0, C(z, 0, s(z))) = C(z, s_0, C(z, 0, t(z))).$$

Symmetry of $D_s(z)$ followed by this equation and $D_t(z)$ yields $s(z) = t(z)$, that is, $E(z)$.

Weaken the original step derivation π into this enlarged retained context, and cut the just-derived $E(z)$ into its distinguished hypothesis. The result is $\Delta, H(z) \vdash E(Sz)$. No substitution into π has occurred: its induction variable is still the same z. Successor computation on each side of $H(Sz)$ then converts this to

$$\Delta, H(z) \vdash H(Sz).$$

Use an unused witness coordinate and witness 0 in primitive progressive DI. The conclusion is $\Delta \vdash D_{x,y} H(y)$; DE returns $\Delta \vdash Q_y H(y)$ because the witness is unused. Reflexivity proves $H(0) = (0 = 0)$, and QE at Sz gives $H(Sz)$, hence $E(Sz)$. We have therefore constructed

$$\Gamma, E(0) \vdash E(Sz).$$

Finally rule (1) discharges the retained base and gives $\Gamma \vdash Q_y E(y)$. □

The complete dependency order is especially short: normal-substitution identities; weakening and cut; elementary equality; unrestricted case DI; case-derived closure; the two decoders; general equation progress. Arbitrary numerical substitution on derivations can consequently be proved afterward, or transferred through the exact comparison with the seven-rule zero-normal calculus. The decoder proof does not use that structural theorem as an unproved prerequisite.

Relation to the executable translation. In `proof_translations.py`, `decoder_closure` constructs the two case-derived closure proofs. In `qi_to_six`, their QE instances are `dl` and `dr`; `decoded` is the congruence-and-transitivity derivation of $E(z)$; and `cut_tree` inserts it into the supplied step proof. The final two operations are guarded progress for H, then case-derived closure for E. Proof-tree numerical lifting merely renames free parameters away from new eigenvariables. It is not arbitrary term substitution into the input step derivation.

2.4 Exact equivalence and bi-interpretation

Theorem 2.5 (Same normal sequents)**.** *For every context* Γ *and formula* A *of the zero-normal language,*

$$\Gamma \vdash_{\mathcal{R}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{Z}} A.$$

Both transformations are effective on finite derivations and are the identity on the sequent syntax.

Proof. For $\mathcal{R} \to \mathcal{Z}$, the first five rules are shared. A case-selected DI is native case introduction. If DI is progress-selected, its matrix must be an equation. Apply the native equation-progress rule to $E(y) := B(t, y)$ to obtain $Q_y E(y)$. Under the temporary assumption $E(0)$, QE supplies $E(Sz)$. The native D case rule discharges $E(0)$ and produces the required $D_{x,y}B$. Hence every primitive rule is sound in $\mathcal{Z}$.

For $\mathcal{Z} \to \mathcal{R}$, translate recursively, retain the five shared rules, replace case introduction by Lemma 2.1, and replace equation progress by Theorem 2.4. Each construction preserves the retained context and its freshness conditions. □

Compose this exact normal-sequent equivalence with the repository's $\mathcal{Z}$–native normalization comparison and the established native–iPRA bi-interpretation. This yields a bi-interpretation and equiconsistency with full first-order iPRA. Both numerical domains remain whole and interpreted equality remains actual equality; the internal numerical round-trip graph is $x = y$. Formula round trips are the inherited round trips composed with the provable equivalence of a formula and its zero-normal form. The normalization layer is explicitly part of the presentation, and is included in any checker measurement.

2.5 The same guard on raw syntax

Proposition 2.6 (Seven rules without normalization)**.** *On the original raw iterator syntax, retain Hyp, I0, IS, Tr, QE and DE, and replace the two introductions by* DI_{R}*, with* $\mathsf{R}(B, t)$ *now testing the unnormalized literal syntax of* $B(t, 0)$*. The resulting seven-rule calculus derives exactly the sequents of the original eight-rule native kernel.*

Proof. Use ordinary capture-free raw substitution throughout. Reflexivity comes from I0 and crossed Tr: apply identity-template transport to the equation $I(0, r; u.u) = r$ twice. Weakening and cut are direct inductions, every missing case introduction is recovered by reflexivity and cut, and true guards are preserved by substitution. The false-to-true repair therefore proves arbitrary substitution exactly as above. The case-derived closure rule is recovered exactly as above.

The one modification is the lift used to recover unrestricted progress. For $E(y) = (s(y) = t(y))$, define the asymmetric equation

$$K(y) := \big(C(y, t(y), s(y)) = C(0, t(y), s(y))\big), \qquad C(v, a, b) := I(v, a; u.b),$$

with each accumulator binder fresh. Its raw zero instance is literally

$$C(0, t(0), s(0)) = C(0, t(0), s(0)),$$

so it satisfies the reflexive-base guard before any computation. For every v, IS computes the left side of $K(Sv)$ to $s(Sv)$, while I0 computes its right side to $t(Sv)$. Elementary equality therefore gives $K(Sv) \dashv\vdash E(Sv)$.

Under the retained base $s(0) = t(0)$, the raw decoder equations are

$$C(y, s(0), C(y, t(y), s(y))) = s(y),$$
$$C(y, s(0), C(0, t(y), s(y))) = t(y).$$

Their zero cases follow from I0 and respectively reflexivity or the retained base. Their successor cases follow from IS for the outer selector and IS or I0 for the inner selector. Case-derived closure and QE supply both decoders. Congruence of $K(y)$ under $C(y, s(0), -)$ therefore yields $E(y)$. Cut this into the unchanged supplied progress proof, convert $E(Sy)$ to $K(Sy)$, and finish with primitive progress followed by case-derived closure, as in Theorem 2.4. The proof translations then give the same raw sequents without a normalization map. □

The asymmetric lift also works in the zero-normal presentation, where its base contracts to $t(0) = t(0)$. Thus the broad reflexive-base guard is independent of the choice to make zero computation definitional. This raw proof does not establish the narrower exact-$0 = 0$ variant on raw syntax: the displayed raw base is reflexive but is generally not literally $0 = 0$.

2.6 Further variants and the implementation tradeoff

Exact-zero base. One may replace $\mathsf{R}(B, t)$ by the smaller class

$$\mathsf{Z}(B, t) \quad \Longleftrightarrow \quad B(t, 0) \text{ is literally } 0 = 0.$$

Every proof above remains valid: selected bases are provable, a true guard is preserved by substitution, and every gate H in the progress proof has exactly this base. Thus this variant also derives precisely the native zero-normal sequents. It uses a single comparison with a fixed formula, though it recognizes fewer usable progress matrices.

Testing before the witness is inserted. Alternatively test whether $B(x, 0)$, with x still a formal variable, is reflexive. The same argument works: a true guard makes every $B(t, 0)$ reflexive, arbitrary gate matrices still qualify, and true guards remain true under substitution of the retained parameters. This removes the witness from the test but usually costs an extra normal-substitution pass in a checker, because the instantiated base $B(t, 0)$ must still be constructed for the unguarded local assumption. The post-witness test above reuses that already-needed base.

Scope of the contribution. The guarded-introduction completeness method is inherited from the repository's diagonal-guard package. The new specialization tests a computed zero instance rather than an exact iterator shape or a literal witness, and its structural proof explicitly handles a guard that can change from false to true. It is not a new arithmetic bridge or a proof of global minimality. A reduction of mathematical bookkeeping does not by itself imply a new record in all checker-size coordinates.

A distinct inherited rule must not be claimed as new. Retaining both $B(t,0)$ and, at equations, $B(t,z)$ in a single introduction premise is another short equivalent presentation. That cumulative introduction already appears as the old DI in the inherited Choice-DI material. It may be used as a control or composed with zero normalization, but it is not a new result of this section.

3 The interpretations and both composites

The transfer to iPRA is separate from the new same-sequent theorem. Bi-interpretability is not inferred from equiconsistency alone.

3.1 The established scalar bridge

The intermediate theory T_{eq} uses ordinary intuitionistic first-order logic with equality over the scalar terms. Its arithmetic assumptions are the two iterator computations, coverage

$$\forall x\,\big(x = 0 \vee \exists y\, x = Sy\big),$$

separation $0 = S0 \to \bot$, and contextual induction for literal equations. The repository proves an interpretation of the native calculus into this theory using the displayed readings of Q, D, and a reverse interpretation using native logical macros. Five inherited mathematical source files and all thirty-nine current Lean modules are supplied as reference material.

For the scalar–iPRA comparison, each fixed iterator term compiles structurally into a fixed primitive-recursive definition. The reverse construction obtains predecessor, a numerical characteristic of equality, quantifier-free induction by Boolean reflection, pairing and projections, and finally a compiler for arbitrary primitive recursion. This order matters: pairing is not assumed while deriving predecessor or equality reflection. Both term translations have provably identity composites. The upstream statements include `biInterp`, `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, and `pra_native_roundTrip` [4].

The compiler acts on each fixed finite function description. The theory does not posit a primitive-recursive universal evaluator for all primitive-recursive programs.

3.2 Explicit composition for the new theory

Let J be the established native-to-iPRA translation and K the established translation back. On terms of the new calculus, J is the same scalar compilation. On formulas its readings are

$$\begin{aligned} J(s = t) &= (J(s) = J(t)),\\ J(Q_y B) &= J(B(0)) \to \forall y\, J(B(y)),\\ J(D_{x,y} B) &= \exists x\big(J(B(x,0)) \to \forall y\, J(B(x,y))\big). \end{aligned}$$

Define the reverse translation into the new syntax by

$$K_{\mathcal{R}}(\varphi) = \mathrm{fnf}(K(\varphi)).$$

Normalization erases literal-zero iterator counters, componentwise through formulas. It does not change a numerical variable or restrict a quantifier's domain.

The existing normalization theorem is

$$\mathrm{fnf}(\Gamma) \vdash_{\mathcal{Z}} \mathrm{fnf}(A) \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathcal{N}} A.$$

Its Lean statement is `nzprf_iff_nprf`; the direct iPRA corollary is `nzprf_iff_pra` [3]. Composition with the new theorem gives, for arbitrary contexts and free parameters,

$$\Gamma \vdash_{\mathcal{R}} A \iff J\Gamma \vdash_{\mathrm{iPRA}} J(A),$$
$$\Delta \vdash_{\mathrm{iPRA}} \varphi \iff K_{\mathcal{R}}\Delta \vdash_{\mathcal{R}} K_{\mathcal{R}}(\varphi).$$

For the native composite, normalize the established interderivability of $KJ(A)$ and A, then apply the new same-sequent theorem. Since A is normal, this proves

$$K_{\mathcal{R}}J(A) \dashv\vdash_{\mathcal{R}} A.$$

For the other composite, normalization correctness identifies $J(\mathrm{fnf}\, K(\varphi))$ with $JK(\varphi)$ inside iPRA, and the inherited round trip identifies the latter with φ. Thus

$$\mathrm{iPRA} \vdash \varphi \leftrightarrow JK_{\mathcal{R}}(\varphi).$$

Both numerical interpretations use the whole domain and actual equality, and their composite isomorphism has graph $x = y$. Finally $0 = S0$ is already normal, so the inherited inconsistency correspondence yields

$$\mathcal{R} \vdash 0 = S0 \quad \iff \quad \mathrm{iPRA} \vdash \bot.$$

4 Complete checker and input contract

The selected checker is the entire standard-library-only module `code/checker.py`. It performs JSON decoding, public syntax validation, capture-avoiding normal substitution, scope checking, all six inference rules, and comparison with the advertised conclusion. It imports no theorem database, proof translator, or trusted helper. The more explicit `code/checker_readable.py` implements the same calculus and input contract.

4.1 Syntax and variables

An input is exactly a JSON object with keys `variables`, `context`, `goal`, and `proof`. `variables` is a nonnegative exact integer. Terms use de Bruijn indices and these tagged arrays:

Mathematics	JSON form
variable i	`["v", i]`
zero	`["z"]`
successor of a	`["s", a]`
iterator	`["i", counter, seed, body]`
equality	`["=", left, right]`
Q-formula	`["q", matrix]`
D-formula	`["d", matrix]`

Indices satisfy $0 \le i < n$ at depth n; Boolean values are not integer indices. An iterator binds one variable only in its body. Q binds one variable, and D binds two, with y at index zero and x at index one. Every literal-zero-counter iterator is rejected by the normal-syntax validator.

The context is a list of formulas in n variables; the goal is another such formula. Every term and formula supplied by a proof instruction is checked at its appropriate depth. The DI and DE eigenvariable conditions are enforced by this representation. Certificates are finite data; no input is evaluated as Python.

4.2 The six instructions

Instruction	Checked meaning
`["hyp", i]`	Return validated context entry i.
`["succ", c, a, h]`	Return the normal successor computation equation.
`["tr", h, p, q]`	Infer $s = t$ from p, check $h(s) = u$ from q, and return $u = h(t)$.
`["qe", t, p, q]`	Infer QB from p, check $B(0)$ from q, and return $B(t)$.
`["de", A, p, q]`	Infer DB from p. Check q under the opened QB and shifted context in one extra variable; its conclusion must be the shifted A. Return A.
`["di", B, t, p]`	Compute $A(y) = B(t, y)$ and its normal zero instance. The local hypothesis is $A(y)$ when that base is reflexive, and the shifted base otherwise. Check p against $A(Sy)$ under that hypothesis and the shifted context. Return DB.

Acceptance is exit status zero with exactly `ACCEPT` and a newline. Invalid syntax, a failed proof obligation or a mismatched conclusion is rejected with nonzero status. The checker uses no Python `assert`, so optimized execution retains its checks. Mathematical completeness concerns finite derivations with unbounded resources; an actual process has stack and memory limits.

4.3 Why the short base test is exact

A validated equality has three fields `["=", r, s]`, whereas Q, D have two. Thus

```
h[1:2] == h[2:]
```

is true exactly for a reflexive equation: it compares `[r]` with `[s]` at equality, and a nonempty singleton with the empty list at a binder. This is justified by complete grammar validation; it is not a test on unrestricted JSON.

The new DI branch first computes `h = shift(sub(a, ["z"]))`, then selects the hypothesis with that test. Its shift is an injective renaming, so it preserves and reflects syntactic reflexivity. Every other branch is unchanged from the matching control.

4.4 Soundness and completeness of the checker interface

Induction on an accepted instruction tree proves its conclusion well-scoped, normal, and derived by the corresponding displayed rule. The substitution walk increases its cutoff exactly at bound coordinates and contracts a rebuilt iterator exactly when its counter is zero. This implements the specified normal substitution. The DI discriminator is exact by the preceding argument; the shifted contexts in DI and DE ensure eigenvariable separation.

Conversely, a finite derivation in the displayed calculus is encoded recursively by these instructions and context indices. Weakening, cut, symmetry, cases and general equation induction are proof-producing macros, not extra trusted instructions. The structural derivation supplies the universal argument; executable tests check its implementation.

5 Measurements and verification

The reference runtime is CPython 3.12.14. Source normalization parses the entire module with `ast.parse`, prints the tree with `ast.unparse`, and appends one linefeed. Significant tokens exclude comments, indentation, dedentation and newline markers. AST nodes count every `ast.walk` node. Gzip uses level nine and timestamp zero, including the wrapper; it is informational.

Complete checker	Rules	Tokens	AST	Bytes	Gzip-9
Zero-normal kernel	7	1073	1136	3272	996
Witness-selected control	6	1013	1075	3095	977
Reflexive-base proposal	6	1013	1073	3084	981

The selected matched comparison saves eleven normalized bytes (0.36%) and two AST nodes (0.19%). Tokens are unchanged and gzip grows by four bytes. The explicit tag-and-sides version is slightly larger than the witness control; the small measured saving uses validated constructor arities. Mathematical bookkeeping and source size should therefore be assessed separately.

The seven-rule zero-normal control connects the certificate translations to the directly formalized kernel. Although it is larger, its rule-count difference is not advertised as new: six rules had already been attained by the witness presentation.

The executed suite passed 13 substantive translation and control cases, 16 malformed-input controls, and 86 standalone process checks, including optimized Python. A further 45 checks compared the selected, readable and source-normalized implementations on the delivered guarded certificates. All expected outcomes agreed.

The examples exercise an actually used induction hypothesis, a nonreflexive induction base, a nested binder, retained outer assumptions, nonzero-witness progress, and literal-zero-witness cases. Two substitution tests change the guard from false to true: variable identification and zero contraction. Naive syntax-only substitution is rejected; the elaborated substitution is accepted. Both translation directions check their input and output with the corresponding kernels and preserve the complete sequent.

These finite checks support the implementation; they do not replace the structural equivalence proof. Complete results and actual certificates are included.

Upstream reference files were authenticated against their Git blob hashes. The included Lean source is reference material, not a new standalone project. This report does not claim a new Lean/Rocq formalization of the reflexive-base rule, a formal verification of the Python checker, or a fresh rebuild of the inherited Lean development.

6 Further research questions

1. Can the same-sequent theorem be formalized directly over `NativeZeroNormal.lean`, preserving the short structural bootstrap and the explicit repair when substitution changes the mode?
2. Can the raw seven-rule version lower the cost of a complete compressed checker? Its asymmetric gate avoids normalization, but testing a base instance still costs code.
3. Which recognizable classes of uniformly provable bases offer the best tradeoff between checker size and proof length? Exact $0 = 0$ and arbitrary $r = r$ already give different choices.
4. Can the certificate translation be shortened while retaining arbitrary contexts, open terms and explicit freshness checks?

5. Can another arithmetic or logical primitive be removed with a net reduction in the complete checking algorithm? Moving computation into a more powerful normalizer alone does not establish that result.

7 Reproduction and deliverables

From the extracted directory:

```
python3 reproduce.py
python3 code/checker.py < tests/certificates/reflexivity.json
python3 build_report.py
```

The first command recomputes measurements and executable checks. The last assembles the standalone `report.tex` and compiles its PDF with pdfLaTeX. All Python uses only the standard library; document creation requires TeX Live. The delivered `report.tex` also compiles directly and contains all its text, tables and code listings.

The archive includes the report, LaTeX, complete checkers and matched controls, certificate translations and examples, verification and measurement scripts, evidence records, source provenance, inherited mathematical appendices and Lean modules, and a checksum manifest.

A The entire selected checker

This is the complete normalized acceptance program; no omitted helper is trusted by it.

```
import json
import sys

def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')

def term(a, n):
    match a:
        case ['v', i]:
            require(type(i) is int and 0 <= i < n)
        case ['z']:
            pass
        case ['s', b]:
            term(b, n)
        case ['i', c, b, h]:
            require(c != ['z'])
            term(c, n)
            term(b, n)
            term(h, n + 1)
        case _:
            raise ValueError('Invalid term')

def formula(a, n):
    match a:
        case ['=', s, t]:
            term(s, n)
            term(t, n)
        case ['q', b]:
            formula(b, n + 1)
        case ['d', b]:
            formula(b, n + 2)
        case _:
            raise ValueError('Invalid formula')

def walk(a, f, d=0):
    if a[0] == 'v':
```

```
        return f(a[1], d)
    a = [a[0]] + [walk(b, f, d + {'i': [0, 0, 1], 'q': [1], 'd': [2]}.get(a[0], [0] * len(a))[j]) for j,
      b in enumerate(a[1:])]
    return a[2] if a[:2] == ['i', ['z']] else a

def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])

def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop])

def infer(p, g, n):
    match p:
        case ['hyp', i]:
            require(type(i) is int and 0 <= i < len(g))
            return g[i]
        case ['succ', c, a, h]:
            term(c, n)
            term(a, n)
            term(h, n + 1)
            return ['=', ['i', ['s', c], a, h], sub(h, ['i', c, a, h])]
        case ['qe', t, p, q]:
            term(t, n)
            a = infer(p, g, n)
            require(a[0] == 'q')
            require(infer(q, g, n) == sub(a[1], ['z']))
            return sub(a[1], t)
        case ['de', a, p, q]:
            formula(a, n)
            b = infer(p, g, n)
            require(b[0] == 'd')
            require(infer(q, [['q', b[1]]] + [shift(x) for x in g], n + 1) == shift(a))
            return a
        case ['di', b, t, p]:
            formula(b, n + 2)
            term(t, n)
            a = sub(['q', b], t)[1]
            h = shift(sub(a, ['z']))
            require(infer(p, [a if h[1:2] == h[2:] else h] + [shift(x) for x in g], n + 1) == sub(a, ['s'
     , ['v', 0]], 0))
            return ['d', b]
        case ['tr', a, p, q]:
            term(a, n + 1)
            e = infer(p, g, n)
            f = infer(q, g, n)
            require(e[0] == f[0] == '=' and f[1] == sub(a, e[1]))
            return ['=', f[2], sub(a, e[2])]
        case _:
            raise ValueError('Invalid proof')

def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and (type(g) is list))
    for b in g:
        formula(b, n)
    formula(a, n)
    require(infer(p, g, n) == a)
if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic.* Snapshot `d9383325b18d349b8bfdc54ef564c5784cc029ec`. https://github.com/VladimirReshetnikov/ProveIt/blob/d9383325b18d349b8bfdc54ef564c5784cc029ec/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] ProveIt, *One computation axiom for iPRA*, package `Research/zero_normal`, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/tree/d9383325b18d349b8bfdc54ef564c5784cc029ec/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/zero_normal.

[3] ProveIt, `ZeroNormalization.lean` and `NativeZeroNormal.lean`, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/d9383325b18d349b8bfdc54ef564c5784cc029ec/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeZeroNormal.lean.

[4] ProveIt, `NativeBiInterpretation.lean`, `NativeRoundTrip.lean`, and `BiInterpretation.lean`, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/d9383325b18d349b8bfdc54ef564c5784cc029ec/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeBiInterpretation.lean.

[5] ProveIt, `inherited_choice_basis.tex`, same snapshot. Contains the older cumulative rule and the logical bridge. https://github.com/VladimirReshetnikov/ProveIt/blob/d9383325b18d349b8bfdc54ef564c5784cc029ec/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/certified_zero/reference/inherited_choice_basis.tex.

[6] ProveIt, package `Research/diagonal_guard`, same snapshot. The general guarded-introduction strategy is inherited from this package. https://github.com/VladimirReshetnikov/ProveIt/tree/d9383325b18d349b8bfdc54ef564c5784cc029ec/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research/diagonal_guard.